## A. HTML

```
<!DOCTYPE html>
<html><head><title>Page</title></head><body>
<h1>Heading</h1> <p>Text<br>next line</p> <a href="b.php">Link</a>
<form method="POST" action="result.php"> <!-- "" = same file -->
  <label for="nm">Name:</label> <input type="text" id="nm" name="nm">
  <input type="password" name="pw">   <input type="email" name="mail">
  <input type="number" name="n" min="0" step="any">
  <input type="radio" name="area" value="Urban"> Urban
  <input type="radio" name="area" value="Rural"> Rural
  <input type="checkbox" name="sub[]" value="PHP"> PHP
  <select name="city"><option value="">-- Select --</option>
                      <option value="Dhaka">Dhaka</option></select>
  <textarea name="note" rows="3"></textarea>
  <input type="submit" name="go" value="Submit">
</form>
<button onclick="calc()">Calculate</button>   <div id="out"></div>
<table border="1"><tr><th>Name</th><th>Marks</th></tr>
                  <tr><td>Karim</td><td>85</td></tr></table>
<script src="script.js"></script>     <!-- last line of the body -->
</body></html>
```

**id**: JavaScript finds the element with `document.getElementById("nm")` (unique, never submitted). **name**: the key PHP receives, `$_POST["nm"]` (no name = not sent; a radio group shares one name; `sub[]` arrives as an array). CSS: `#out { color:#000; background:#eee; margin:4px; padding:4px; border:1px solid #000; }`

## B. JAVASCRIPT

### Variables, input and output

```
let n = 5;  const GOAL = 20;  let s = "Hi", ok = true, arr = [1, 2];
parseInt("42") 42  parseFloat("3.5") 3.5  Number("7") 7  String(7) "7"
(2.456).toFixed(2) "2.46" (string)    isNaN(x) true if not a number
alert("Saved");  console.log(x);  let age = prompt("Age?"); // text
let h = parseFloat(document.getElementById("hours").value);   // read
document.getElementById("out").innerHTML = `Total: ${n}`;   // show
document.getElementById("out").innerHTML += "<br>one more line";
document.getElementById("hours").value = "";                // clear
document.getElementById("urban").checked    // radio: true/false
```

### Operators and conditions

```
+ - * / % **  == === != !== < > <= >=  && || !  ++ --  = += -= *= /=
"5" + 2 -> "52"  "5" - 2 -> 3  5 === "5" false  5 == "5" true
if (t >= 20) msg = "Goal reached!";     // ranges: test from the top
else if (t > 12) msg = "Almost ready!";
else msg = "Start!";
if (loc === "Dhaka") { rate = urban ? 7.5 : 6.5; }  // nested, ternary
switch (g) { case "A": m = "Top"; break;  default: m = "Other"; }
```

### Loops

```
for (let i = 1; i <= 10; i++) { sum += i; }          // summation
while (n > 0) { d += n % 10; n = Math.floor(n / 10); } // digit sum
do { k++; } while (k < 3);                            // at least once
for (let v of arr) { if (v < 0) continue; if (v > 99) break; c++; }
```

### Arrays

```
let a = [12, 15, 14];  a[0] = 9;  a.length 3  a[a.length - 1] (last)
a.push(7)  a.pop()  a.includes(15)  a.indexOf(15) (-1 = not found)
a.join(", ") "9, 15, 14"    let sum = 0; for (let v of a) sum += v;
let avg = sum / a.length;   Math.max(...a)   Math.min(...a)
a.reduce((s, v) => s + v, 0)  a.filter(v => v >= 40)  a.map(v => v*2)
```

**Numeric sort.** Default `sort()` sorts as strings.

```
a.sort()                  // as STRINGS: [10, 9, 100] -> [10, 100, 9]
a.sort((x, y) => x - y)   // numbers ascending   [9, 10, 100]
a.sort((x, y) => y - x)   // numbers descending  [100, 10, 9]
```

### Strings

```
s.length  s.toUpperCase()  s.toLowerCase()  s.trim()  s.endsWith("*")
s.includes("ab")  s.indexOf("a")  s.slice(0, 3)  s.substring(1, 4)
s.replace("a", "b")   s.split(",") -> array   arr.join("") -> string
let rev = s.toLowerCase().split("").reverse().join(""); // palindrome
/[A-Z]/.test(p) /[a-z]/.test(p) /[0-9]/.test(p) /[!@#$%^&*]/.test(p)
"Total: " + t + " h"     `Total: ${t} h`     s1 === s2 (same text)
```

### Functions

```
function average(total, count) { return total / count; } // params
// <button onclick="addStudy()">Add Study</button> calls it on click
function addStudy() {
  let h = parseFloat(document.getElementById("hours").value);
  document.getElementById("out").innerHTML = average(h, 2); // args
}
```

### Math and random

```
Math.round(4.5) 5  Math.floor(4.9) 4  Math.ceil(4.1) 5  Math.abs(-3) 3
Math.sqrt(16) 4  Math.pow(2, 3) 8  Math.max(1, 5) 5  Math.min(1, 5) 1
Math.random() 0 to <1.  Whole number from min to max (both included):
Math.floor(Math.random() * (max - min + 1)) + min    // 500..5000
```

## F. RULES THAT COST MARKS

- JS `.value` is a string: `parseInt`/`parseFloat` first ("2"+"3" = "23"); compare with `===`.
- Totals, counters and arrays live **outside** the function. Ranges: test from the top with `>=`.
- JS `arr.length`; PHP `count($a)`, `strlen($s)`. JS numbers sort only with a comparator.
- PHP: `$` variables, `.` joins, `;` ends lines; `return` hands a value back, `echo` prints. Whole units: `ceil(need / per)`.
- Forms: `method="POST"` + a `name` on every field; check `isset($_POST[...])`.
- SQL text in single quotes, apostrophe doubled: `'Cox''s'`. Use the paper's exact database, table, column names.
- `UPDATE`/`DELETE` need the right `WHERE` (try it as SELECT first). Aggregate conditions: `HAVING`.
- Submitted values: `prepare` + `bind_param`. Ratings, money: `DECIMAL`, not `FLOAT`.

## C. PHP

```
<?php echo "Hi"; ?>   <?= $x ?> short echo   // statements end with ;
$name = "Karim";  $n = 5;  $f = 2.5;      // variables start with $
echo "Hi $name<br>";  echo 'Hi ' . $name;  $s .= "!";  // . joins text
(int) "42"  (float) "2.5"  7 % 2 -> 1  intdiv(7, 2) -> 3  abs(-3)
ceil(4.1) 5  floor(4.9) 4  round(25.639, 2) 25.64  max(1, 9)  min()
number_format(15000) "15,000"  number_format(2.5, 2) "2.50"  rand(1,6)
if ($t >= 120) { } elseif ($t >= 90) { } else { }
switch ($c) { case "A": $m = "Top"; break;  default: $m = "Other"; }
for ($i = 1; $i <= 10; $i++) { }  while ($n > 0) { }  do { } while ();
$a = [3, 1, 2];  $a[] = 4;  count($a)  array_sum($a)  max($a)  min($a)
$m = ["Dhaka" => 7.5, "Other" => 6.5];  echo $m["Dhaka"];
foreach ($a as $v) { }      foreach ($m as $city => $rate) { }
function area($w, $h) { return $w * $h; }   echo area(2, 3);   // 6
```

### Array and string functions

```
sort rsort    values asc/desc, keys renumbered (numbers as numbers)
asort arsort  by value asc/desc, keys kept     ksort krsort  by key
in_array(4, $a)  isset($x) set, not null  empty($x) "", 0, [], null
strlen strtolower strtoupper trim ucfirst strrev str_repeat("*", 3)
strpos($s, "x") position or false   substr($s, 0, 3)  substr($s, -1)
str_replace("a", "b", $s)  explode(",", $s) array  implode(", ", $a)
preg_match('/[A-Z]/', $p)  1 if $p contains an uppercase letter
```

### Form, POST only

```
<form method="POST" action="">                  <!-- POST only -->
  <input type="number" name="n"> <input type="text" name="nm">
  <input type="submit" name="go" value="Go"></form>
<?php
if (isset($_POST["go"])) {                // only after a submit
  $n = (int) $_POST["n"];
  $nm = trim($_POST["nm"] ?? "");      // "" if the field is missing
  if ($n <= 0 || $nm === "") echo "<p>Invalid input</p>";
  else echo "<p>" . htmlspecialchars($nm) . ": $n</p>"; // safe output
}
?>
```

## D. SQL (ONE EXAMPLE SCHEMA FOR EVERY QUERY)

```
CREATE DATABASE uiu;   USE uiu;
CREATE TABLE departments (dept VARCHAR(10) PRIMARY KEY,
  dname VARCHAR(60));
CREATE TABLE students (id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL, dept VARCHAR(10), cgpa DECIMAL(3,2),
  credits INT, city VARCHAR(30),
  FOREIGN KEY (dept) REFERENCES departments(dept));
INSERT INTO departments VALUES ('CSE', 'Computer'), ('EEE', 'Elec');
INSERT INTO students (name, dept, cgpa, credits, city) VALUES
  ('Karim','CSE',3.75,90,'Dhaka'), ('Rahim','EEE',2.95,60,NULL);
ALTER TABLE students ADD email VARCHAR(60);        -- add a column
```

```
SELECT name, cgpa FROM students WHERE cgpa >= 3.5;  -- * = all columns
SELECT DISTINCT city FROM students;                  -- each value once
SELECT * FROM students ORDER BY cgpa DESC, name ASC LIMIT 3;  -- top 3
... WHERE city = 'Dhaka' AND cgpa > 3   -- both true; OR: one; NOT
... WHERE dept IN ('CSE', 'EEE')        -- one of a list
... WHERE credits BETWEEN 45 AND 75     -- range, both ends included
... WHERE name LIKE 'K%'   -- starts with K; '%im%' contains; _ 1 char
... WHERE city IS NULL                  -- empty; IS NOT NULL has one
SELECT COUNT(*), SUM(credits), AVG(cgpa), MIN(cgpa), MAX(cgpa)
  FROM students;                              -- one row of totals
SELECT dept, COUNT(*) AS n FROM students GROUP BY dept;  -- per group
SELECT dept, AVG(cgpa) AS a FROM students WHERE credits > 30
  GROUP BY dept HAVING AVG(cgpa) > 3.0 ORDER BY a DESC;
  -- WHERE filters rows BEFORE grouping; HAVING filters the groups
SELECT s.name, d.dname FROM students s
  INNER JOIN departments d ON s.dept = d.dept;       -- matches only
SELECT d.dname, COUNT(s.id) AS n FROM departments d
  LEFT JOIN students s ON s.dept = d.dept
  GROUP BY d.dept, d.dname;                  -- every dept, 0 if no student
UPDATE students SET cgpa = 3.80 WHERE id = 1;  -- change existing rows
UPDATE students SET credits = credits * 1.10 WHERE dept='CSE'; -- +10%
UPDATE students SET credits = credits + 5
  WHERE credits > 80 AND credits + 5 <= 90;    -- only if result <= 90
UPDATE students SET cgpa = CASE WHEN cgpa > 3.5 THEN cgpa - 0.1
  ELSE cgpa + 0.1 END;                       -- two rules, each row once
UPDATE students SET credits = credits + 3 ORDER BY id DESC LIMIT 1;
DELETE FROM students WHERE id = 2;    -- remove rows (no WHERE = all!)
DROP TABLE students;                   -- removes the table itself
SELECT name, CASE WHEN cgpa >= 3.5 THEN 'Top' ELSE 'Regular' END
  AS label FROM students;                          -- label each row
SELECT name FROM students s WHERE cgpa >
  (SELECT AVG(cgpa) FROM students WHERE dept = s.dept); -- own average
SELECT name, credits * 100 / (SELECT SUM(credits) FROM students)
  AS pct FROM students;                            -- % of total
```

## E. PHP + MYSQLI + XAMPP

XAMPP: **Start Apache + MySQL**; `localhost/phpmyadmin` (SQL tab: paste CREATE/INSERT; Export: .sql). PHP file in `htdocs/final/`, open `http://localhost/final/q3.php`, never `file:///`. User `root`, password "".

```
<?php
$conn = new mysqli("localhost", "root", "", "uiu"); // host,user,pw,db
if ($conn->connect_error) die("Failed: " . $conn->connect_error);
$res = $conn->query("SELECT name, cgpa FROM students");  // fixed SQL
echo "<table border='1'><tr><th>Name</th><th>CGPA</th></tr>";
while ($row = $res->fetch_assoc())        // $res->num_rows = rows
  echo "<tr><td>" . htmlspecialchars($row["name"]) . "</td><td>"
     . $row["cgpa"] . "</td></tr>";
echo "</table>";
$city = $_POST["city"] ?? "";              // submitted value: prepare
$stmt = $conn->prepare("SELECT name FROM students WHERE city = ?");
$stmt->bind_param("s", $city);          // s string, i int, d decimal
$stmt->execute();  $res = $stmt->get_result();  // then fetch_assoc()
$stmt = $conn->prepare(
  "INSERT INTO students (name, dept, cgpa) VALUES (?, ?, ?)");
$stmt->bind_param("ssd", $nm, $dp, $cg); $stmt->execute(); // new id
$stmt = $conn->prepare("UPDATE students SET cgpa = ? WHERE id = ?");
$stmt->bind_param("di", $cg, $id);  $stmt->execute(); // affected_rows
$stmt = $conn->prepare("DELETE FROM students WHERE id = ?");
$stmt->bind_param("i", $id);  $stmt->execute();
$stmt->close();  $conn->close();
```

## Question 1 JavaScript

Final Exam Spring 2026 (261), Set A · [10]

Develop a JavaScript application where users can track their weekly study hours by subject and get feedback. At first, create two input fields: **Subject Name** and **Study Hours**, an **"Add Study"** button, and a label area to display total study hours, average study hours, subjects studied, and feedback. The user will enter the subject name and the number of hours studied in each session. When the button is clicked, the application will do the following:

**a.** Set a weekly study goal of **20 hours**. **b.** Add the entered study hours to a running total. **c.** Store each entered subject name. **d.** Count how many study sessions the user has entered. **e.** Calculate the average study hour so far: **Average = Total Study Hours / Number of Sessions** **f.** Display: total study hours, average study hours, subjects studied in all sessions, feedback message. **g.** Display feedback based on total study hours: 0–5 → "Start studying regularly!", 6–12 → "Good progress!", 13–19 → "Almost ready!", 20+ → "Weekly goal reached!" **h.** If the user enters more than 7 study sessions without reaching 20 hours, display **"Increase your study time!"**

| Sample Input | Sample Output |
|---|---|
| English-2, English-3, Bangla-4, Physics-5 | Total: 14 hours, Average: 3.5 hours, Subjects: [English, English, Bangla, Physics], Almost ready! |
| Math-5, Physics-6, Chemistry-7, English-3 | Total: 21 hours, Average: 5.25 hours, Subjects: [Math, Physics, Chemistry, English], Weekly goal reached! |
| 8 sessions, total 15 hours | Total: 15 hours, Average: 1.875 hours, Increase your study time! |

**Answer. index.html**

```
<!DOCTYPE html>
<html><head><title>Study Tracker</title></head><body>
Subject Name: <input type="text" id="subject"><br><br>
Study Hours: <input type="number" id="hours"><br><br>
<button onclick="addStudy()">Add Study</button><br><br>
<label id="report"></label>
<script src="script.js"></script></body></html>
```

**script.js (a to h marked in the comments)**

```
const GOAL = 20;      // a. weekly goal
let total = 0;        // b. running total
let subjects = [];    // c. subject names
let sessions = 0;     // d. number of sessions
function addStudy() {
  let subject = document.getElementById("subject").value.trim();
  let hours = parseFloat(document.getElementById("hours").value);
  if (subject === "" || isNaN(hours) || hours < 0) {
    document.getElementById("report").innerHTML =
      "Enter a subject and valid hours.";
    return;
  }
  total += hours;                       // b
  subjects.push(subject);               // c
  sessions++;                           // d
  let average = Math.round(total / sessions * 1000) / 1000;   // e
  let feedback;                         // g: test from the top
  if (total >= GOAL) feedback = "Weekly goal reached!";
  else if (total > 12) feedback = "Almost ready!";
  else if (total > 5) feedback = "Good progress!";
  else feedback = "Start studying regularly!";
  if (sessions > 7 && total < GOAL)     // h
    feedback = "Increase your study time!";
  document.getElementById("report").innerHTML =            // f
    `Total: ${total} hours, Average: ${average} hours, ` +
    `Subjects: [${subjects.join(", ")}], ${feedback}`;
}
```

## Question 2 PHP

Final Exam Spring 2026 (261), Set A · [10]

Write a PHP program for a student academic tracking system. The program should take three inputs from the user via an HTML form: **credits earned per course**, **number of courses completed**, **target graduation credits**. First, calculate the **total credits earned** by multiplying the credits per course by the number of courses completed. Classify the academic progress using the following conditions: **Excellent:** total credits 120 or more; **Good:** 90 or more but less than 120; **Average:** 60 or more but less than 90; **Poor:** less than 60. The program should then compare the total credits earned with the target graduation credits. Finally, display the total credits, the progress category, the target value, and clearly state whether the progress is **above**, **below**, or **exactly at** the target, along with the difference.

| Credits/ Course | Courses Completed | Target | Total Credits | Progress | Result |
|---|---|---|---|---|---|
| 4 | 30 | 120 | 120 | Excellent | Target met exactly (0 difference) |
| 3 | 35 | 100 | 105 | Good | Above target by 5 |
| 3 | 25 | 90 | 75 | Average | Below target by 15 |
| 4 | 10 | 60 | 40 | Poor | Below target by 20 |

**Answer. index.php (one file; the form posts to itself)**

```
<!DOCTYPE html><html><body>
<form method="POST" action="">
  Credits/course: <input type="number" name="credits" required><br>
  Courses completed: <input type="number" name="courses" required><br>
  Target credits: <input type="number" name="target" required><br>
  <input type="submit" name="calc" value="Calculate">
</form>
<?php
if (isset($_POST["calc"])) {
  $credits = (int) $_POST["credits"];
  $courses = (int) $_POST["courses"];
  $target = (int) $_POST["target"];
  $total = $credits * $courses;                 // total credits earned
  if ($total >= 120)     $progress = "Excellent";
  elseif ($total >= 90)  $progress = "Good";
  elseif ($total >= 60)  $progress = "Average";
  else                   $progress = "Poor";
  $diff = $total - $target;                     // compare with target
  if ($diff > 0)      $result = "Above target by $diff";
  elseif ($diff < 0)  $result = "Below target by " . abs($diff);
  else                $result = "Target met exactly (0 difference)";
  echo "<table border='1'><tr><th>Total Credits</th>";
  echo "<th>Progress</th><th>Target</th><th>Result</th></tr>";
  echo "<tr><td>$total</td><td>$progress</td><td>$target</td>";
  echo "<td>$result</td></tr></table>";
}
?>
</body></html>
```

## Question 3 PHP & MySQL

Final Exam Spring 2026 (261), Set A · [10]

Create a database named tourism and a table named tourist_spot with the following structure and values:

| SpotID | SpotName | Region | Category | Rating | EntryFee | VisitorsPerYear |
|---|---|---|---|---|---|---|
| 1 | Cox's Bazar Beach | CTG | Beach | 4.9 | 0 | 150000 |
| 2 | Sundarbans Forest | KHL | Nature | 4.3 | 150 | 80000 |
| 3 | Ratargul Forest | SYL | Nature | 4.8 | 100 | 60000 |
| 4 | Kuakata Beach | PTU | Beach | 4.5 | 0 | 90000 |
| 5 | Shat Gambuj Mosque | BGH | Heritage | 4.7 | 30 | 70000 |
| 6 | Paharpur | NWG | Heritage | 4.1 | 30 | 40000 |

Now, do the following **(write full PHP–MySQL code): 1.** Display the SpotName, Region and Rating of all spots whose rating is above 4.5 or whose SpotName contains the word 'Beach'. Sort the result by Rating in descending order. **2.** Update: for all spots whose rating is between 4.0 and 4.5 (inclusive) and whose EntryFee is greater than 0, increase the Rating by 0.2 and raise the EntryFee by 10%. **3.** For each category, show the category name, the number of spots in it, the average rating, and the total visitors per year but only include categories whose average rating is above 4.4. Sort the result by average rating in descending order. **4.** For each region, show the region name and its total estimated yearly revenue, calculated as the sum of (EntryFee × VisitorsPerYear) over its spots. Display only those regions whose total estimated revenue exceeds 1,000,000, ordered from highest to lowest.

**Answer. tourism.sql (phpMyAdmin, SQL tab)**

```
CREATE DATABASE tourism;
USE tourism;
CREATE TABLE tourist_spot (
  SpotID INT PRIMARY KEY,
  SpotName VARCHAR(100),
  Region VARCHAR(10),
  Category VARCHAR(30),
  Rating DECIMAL(3,1),          -- exact decimals (not FLOAT)
  EntryFee DECIMAL(8,2),
  VisitorsPerYear INT
);
INSERT INTO tourist_spot VALUES
(1, 'Cox''s Bazar Beach', 'CTG', 'Beach', 4.9, 0, 150000),
(2, 'Sundarbans Forest', 'KHL', 'Nature', 4.3, 150, 80000),
(3, 'Ratargul Forest', 'SYL', 'Nature', 4.8, 100, 60000),
(4, 'Kuakata Beach', 'PTU', 'Beach', 4.5, 0, 90000),
(5, 'Shat Gambuj Mosque', 'BGH', 'Heritage', 4.7, 30, 70000),
(6, 'Paharpur', 'NWG', 'Heritage', 4.1, 30, 40000);
```

**index.php (tasks 1 to 4; task 2 runs only on its POST button)**

```
<?php
$conn = new mysqli("localhost", "root", "", "tourism");
if ($conn->connect_error)
  die("Connection failed: " . $conn->connect_error);

function show($conn, $sql) {        // prints any SELECT as a table
  $res = $conn->query($sql);
  echo "<table border='1'><tr>";
  foreach ($res->fetch_fields() as $f) echo "<th>$f->name</th>";
  echo "</tr>";
  while ($row = $res->fetch_assoc()) {
    echo "<tr>";
    foreach ($row as $v) echo "<td>" . htmlspecialchars($v) . "</td>";
    echo "</tr>";
  }
  echo "</table>";
}
?>
<!DOCTYPE html>
<html><body>
<h3>1) Rating above 4.5 or name contains Beach</h3>
<?php show($conn, "SELECT SpotName, Region, Rating FROM tourist_spot
  WHERE Rating > 4.5 OR SpotName LIKE '%Beach%'
  ORDER BY Rating DESC"); ?>

<h3>2) Rating +0.2 and EntryFee +10%</h3>
<form method="POST" action="">
  <button type="submit" name="task2">Run task 2 (once)</button>
</form>
<?php
if (isset($_POST["task2"])) {
  $conn->query("UPDATE tourist_spot
    SET Rating = Rating + 0.2, EntryFee = EntryFee * 1.10
    WHERE Rating BETWEEN 4.0 AND 4.5 AND EntryFee > 0");
  echo "<p>" . $conn->affected_rows . " spot(s) updated.</p>";
}
show($conn, "SELECT * FROM tourist_spot");
?>
<h3>3) Categories with average rating above 4.4</h3>
<?php show($conn, "SELECT Category, COUNT(*) AS Spots,
  ROUND(AVG(Rating), 2) AS AvgRating,
  SUM(VisitorsPerYear) AS TotalVisitors
  FROM tourist_spot GROUP BY Category
  HAVING AVG(Rating) > 4.4 ORDER BY AvgRating DESC"); ?>

<h3>4) Regions with revenue above 1,000,000</h3>
<?php show($conn, "SELECT Region,
  SUM(EntryFee * VisitorsPerYear) AS Revenue
  FROM tourist_spot GROUP BY Region
  HAVING Revenue > 1000000 ORDER BY Revenue DESC");
$conn->close(); ?>
</body></html>
```

**Results** (MariaDB 10.4). **1)** Cox's Bazar Beach 4.9, Ratargul Forest 4.8, Shat Gambuj Mosque 4.7, Kuakata Beach 4.5 (name has "Beach"). **2)** 2 rows: Sundarbans 4.3→4.5, fee 150→165.00; Paharpur 4.1→4.3, fee 30→33.00 (Kuakata skipped: fee 0). **3)** after task 2: Beach 2, 4.70, 240000 · Nature 2, 4.65, 140000 · Heritage 2, 4.50, 110000 (before task 2 Heritage is exactly 4.40, not above 4.4). **4)** after task 2: KHL 13,200,000 · SYL 6,000,000 · BGH 2,100,000 · NWG 1,320,000. Press task 2 **once**: a second press adds 0.2 again.

### BEFORE SUBMITTING THESE ANSWERS

- **Q1:** `total`, `subjects`, `sessions` are declared outside `addStudy()`; the ids in `getElementById` match the HTML; 20 hours must give "Weekly goal reached!" (`>= GOAL`).
- **Q2:** `method="POST"` and a `name` on every input; open via `http://localhost/`.
- **Q3:** import `tourism.sql` first; `BETWEEN` includes 4.0 and 4.5; group conditions use `HAVING`; `Rating` is `DECIMAL(3,1)` so 4.40 is not above 4.4.